SAN Assignment - Generalized linear models

Author : Šimon Gavula

Email : gavulsim@fel.cvut.cz

Introduction

The aim of this assignment is to practice constructing linear models. You will start with a simple linear model. You will evaluate and interpret it (1p). Consequently, your task will be to improve this model using generalized linear models (GLMs) and feature transformations. You will get 1p for proposal and evaluation of GLM (family, evaluation, interpretation), 1p for correct feature transformations, 2p for proposal and justification of the final model and eventually, 1p for comprehensive evaluation of all the model improvements (ablation study through cross-validation, note that the previous evaluations must be done without cross-validation).

Submit your solution consisting of both this modified notebook file and the exported PDF/HTML document as an archive to the courseware BRUTE upload system for the SAN course. The deadline is specified there.

Input data

In this assignment, you will work with a student dataset. The dataset contains 200 samples and 4 features: *num_awards* is the outcome variable and indicates the number of awards earned by students in a year, *math* is a continuous predictor variable and represents students' scores on their math final exam, *prog* is a categorical predictor variable with three levels indicating the type of program in which the students were enrolled (1 = "General", 2 = "Academic" and 3 = "Vocational"), and *work* is a continuous predictor that gives the number of hours that students spent at work on average per week.

Load the necessary libraries and the dataset

In [1]:

```python
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
from scipy import stats
from collections import defaultdict
import scipy.stats
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold

print_precision = 3
cut_threshold = 1e5
thr = np.format_float_scientific(cut_threshold)
formatter_short = ("{:." + f"{print_precision}" + "f}").format
cond_formatter = lambda x: formatter_short(x) if np.abs(x) < cut_threshold else f">{thr}"

np.set_printoptions(formatter={'float_kind': cond_formatter})
pd.options.display.float_format = cond_formatter
```

In [2]:

```python
df = pd.read_csv("study_data.csv")
X = df.loc[:, df.columns != "num_awards"]
X = sm.add_constant(X)
y = df.num_awards.to_numpy()
```

In [3]:

```python
X.head()
```

Out[3]:

	const	prog	math	work
45	1.000	3	41	19
108	1.000	1	41	31
15	1.000	3	44	3
67	1.000	3	42	34
153	1.000	3	40	0

In [4]:

```python
y
```

Out[4]:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 1, 0, 0, 0, 3, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1,
       0, 0, 1, 0, 2, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       1, 1, 2, 0, 1, 0, 1, 0, 0, 2, 1, 3, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1,
       0, 0, 0, 0, 3, 1, 0, 0, 0, 3, 0, 0, 1, 1, 0, 0, 0, 3, 0, 0, 0, 2,
       0, 1, 5, 0, 0, 1, 0, 0, 2, 4, 1, 0, 1, 1, 3, 1, 1, 3, 2, 0, 0, 1,
       1, 3, 2, 0, 6, 4, 1, 0, 2, 2, 1, 2, 1, 1, 0, 1, 2, 5, 1, 2, 2, 1,
       0, 3])
```

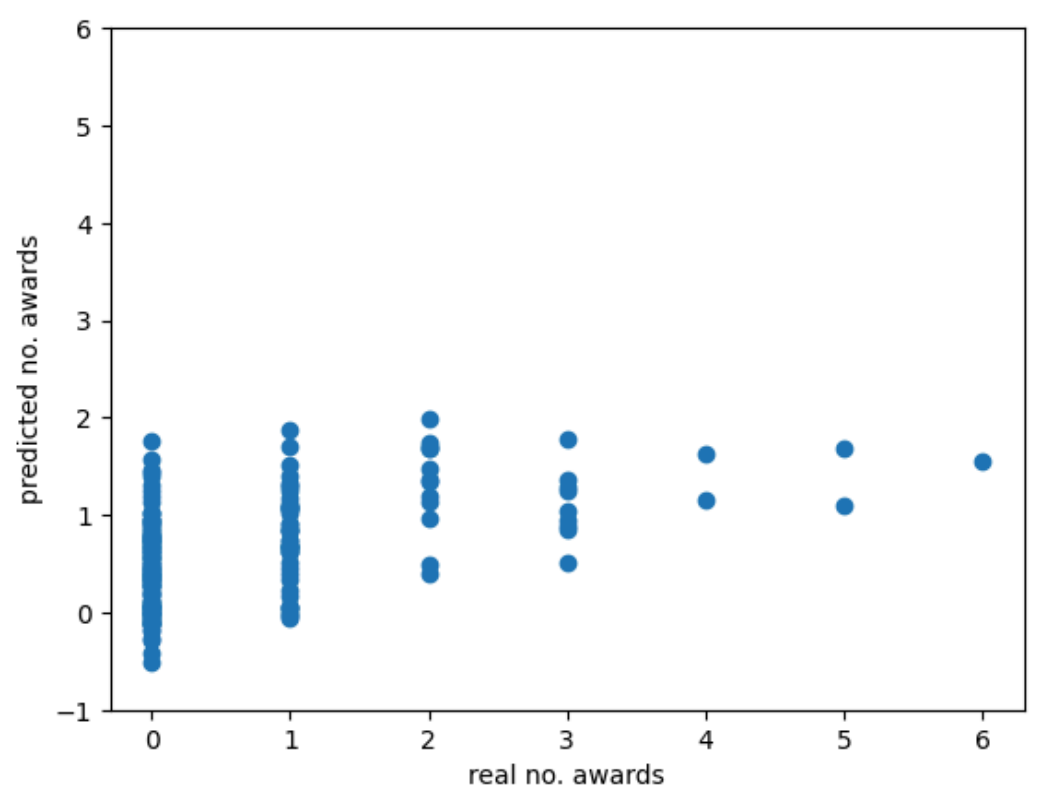
Simple linear model

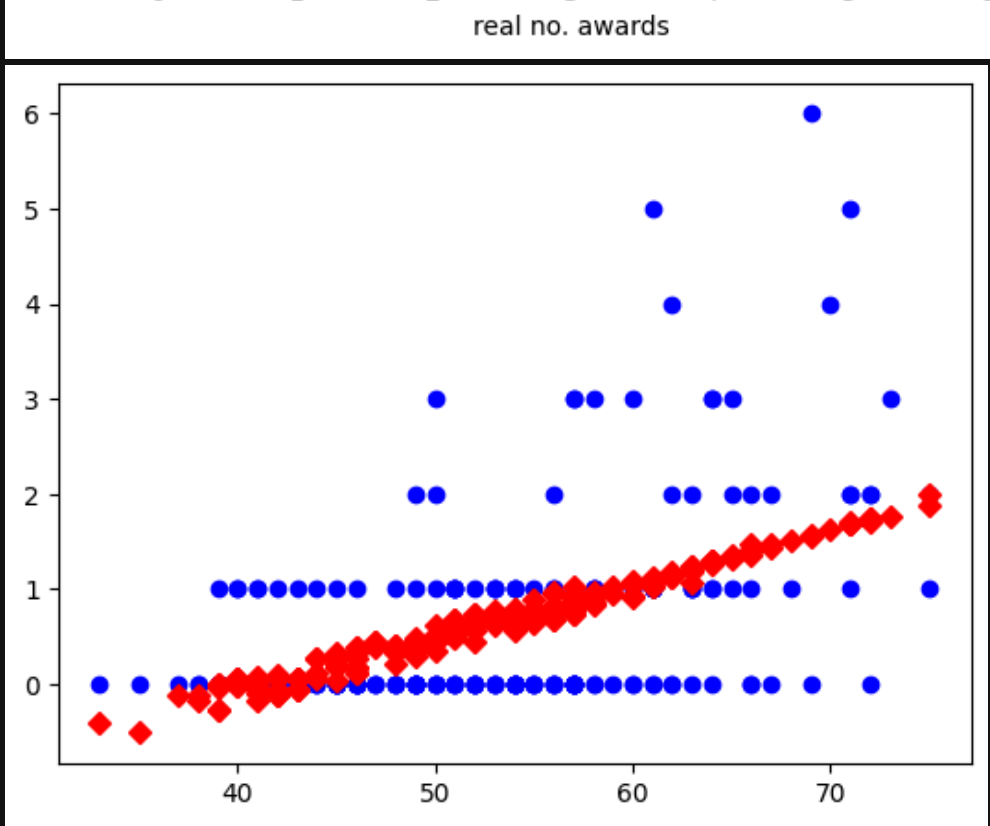
Let us start with an ordinary linear model with no feature transformations. Explain how far the model works (does it meet formal assumptions?, does it overcome the null model?). Which predictors would you keep there and which of them are not useful? Use the standard evaluation procedures that we have for linear models (notice, that GML with Gaussian family corresponds to an OLS model, only the summary is slightly more general).

In [5]:

```python
res = sm.OLS(endog=y, exog=X).fit()
print(res.summary())
print("")
print("--------------------------------------------------------------------------------")
print("")
res = sm.GLM(endog=y, exog=X, family=sm.families.Gaussian()).fit()
print(res.summary())

plt.scatter(y, res.predict(X))
plt.ylabel("predicted no. awards")
plt.xlabel("real no. awards")
plt.ylim((-1, 6))
plt.show()


try:
    sm.graphics.plot_fit(res, "math")
except AttributeError:
    pass
finally:
    plt.show()
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.253
Model:                            OLS   Adj. R-squared:                  0.242
Method:                 Least Squares   F-statistic:                     22.16
Date:                Mon, 10 Nov 2025   Prob (F-statistic):           2.13e-12
Time:                        13:06:25   Log-Likelihood:                -264.40
No. Observations:                 200   AIC:                             536.8
Df Residuals:                     196   BIC:                             550.0
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.5363      0.481     -5.268      0.000      -3.486      -1.587
prog           0.1098      0.096      1.144      0.254      -0.079       0.299
math           0.0566      0.007      7.941      0.000       0.043       0.071
work          -0.0024      0.006     -0.410      0.682      -0.014       0.009
==============================================================================
Omnibus:                       84.747   Durbin-Watson:                   2.052
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              294.684
Skew:                           1.737   Prob(JB):                     1.02e-64
Kurtosis:                       7.826   Cond. No.                         416.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

--------------------------------------------------------------------------------

                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  200
Model:                            GLM   Df Residuals:                      196
Model Family:                Gaussian   Df Model:                            3
Link Function:               Identity   Scale:                         0.84055
Method:                          IRLS   Log-Likelihood:                -264.40
Date:                Mon, 10 Nov 2025   Deviance:                       164.75
Time:                        13:06:25   Pearson chi2:                     165.
No. Iterations:                     3   Pseudo R-squ. (CS):             0.2829
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.5363      0.481     -5.268      0.000      -3.480      -1.593
prog           0.1098      0.096      1.144      0.253      -0.078       0.298
math           0.0566      0.007      7.941      0.000       0.043       0.071
work          -0.0024      0.006     -0.410      0.682      -0.014       0.009
==============================================================================
```





In [6]:

```python
print("AIC score for OLS model: ", res.aic)
```

```
AIC score for OLS model:  536.7950226121017
```

In [7]:

```python
null_model = sm.GLM(endog=y, exog=X.const, family=sm.families.Gaussian()).fit()
null_model.summary()
```

Out[7]:

Generalized Linear Model Regression Results

Dep. Variable:	y	**No. Observations:**	200
Model:	GLM	**Df Residuals:**	199
Model Family:	Gaussian	**Df Model:**	0
Link Function:	Identity	**Scale:**	1.1086
Method:	IRLS	**Log-Likelihood:**	-293.60
Date:	Mon, 10 Nov 2025	**Deviance:**	220.62
Time:	13:06:26	**Pearson chi2:**	221.
No. Iterations:	3	**Pseudo R-squ. (CS):**	1.254e-05
Covariance Type:	nonrobust		

	coef	std err	z	P>|z|	[0.025	0.975]
const	0.6300	0.074	8.462	0.000	0.484	0.776

In [8]:

```python
print("AIC score for NULL model: ", null_model.aic)
```

```
AIC score for NULL model:  589.2002928805089
```

Add your verbal summary and comments here (1p):

Model summary

The ordinary linear regression model explains approximately 25% of the variance in the response variable ($R^2 = 0.253$, adjusted $R^2 = 0.242$). The overall F-statistic (22.16, $p < 2.13e-12$) indicates that at least one predictor significantly contributes to the model.

Individual p-tests checks

Based on individual p-tests:

1. The variable prog has a p-value of 0.254 (> 0.05), and therefore this variable is not significant.
2. The variable math has a p-value of 0.000 (< 0.05), and therefore it is statistically significant.
3. The variable work has a p-value of 0.682 (> 0.05), which is not significant.

Therefore, from individual t-tests we can conclude that only math appears to be a useful predictor in this model and prog variable and work variable can be excluded.

Assumption checks

The dependent variable (number of awards) is a count variable, so the assumption of normally distributed residuals required by OLS does not hold. A Generalized Linear Model is therefore more appropriate.

NULL model vs full ordinary linear regression model

The ols model improves over the null model. The null model deviance 220.62 drops to 164.75 for the full model, and the log-likelihood increases from −293.60 to −264.40. This indicates that including predictors improves the model's fit. The AIC is also lower, indicating a better overall fit for the full ols model.

Generalized linear model

Now, the goal is to implement a generalized linear model that conceptually fits the given task. Do not transform the predictors yet, use them as they are, or omit them from the model. Once you obtain your model, interpret the effect of the *math* predictor on the outcome. How (according to your model) increasing a math score by one point affects the number of awards won?

Explain why the model overcomes the previous linear model, or justify that the generalized model is not needed. Compare the models theoretically as well as technically in terms of a proper quality measure(s). Note: The difference between the models generally cannot be statistically tested.

Because in Python you cannot use *anova* function refer for example to the Likelihood ration test (LTR) when comparing **nested** models, which you can easily implement yourself.

In [9]:

```python
from scipy import stats

def compareGLM(model_simple, model_complex):
    LL_simple = model_simple.llf
    LL_complex = model_complex.llf
    df_diff = model_complex.df_model - model_simple.df_model
    LR_stat = 2 * (LL_complex - LL_simple)
    p_value = stats.chi2.sf(LR_stat, df_diff)

    print("=== GLM Model Comparison (Likelihood Ratio Test) ===")
    print(f"Log-likelihood (simple):   {LL_simple:.3f}")
    print(f"Log-likelihood (complex):  {LL_complex:.3f}")
    print(f"LR statistic:              {LR_stat:.3f}")
    print(f"Degrees of freedom diff:   {df_diff}")
    print(f"p-value:                   {p_value:.5f}")
    print(f"AIC (simple):              {model_simple.aic:.3f}")
    print(f"AIC (complex):             {model_complex.aic:.3f}")
```

In [10]:

```python
# TODO: add your code here
X_just_math = X.copy()
X_just_math = X_just_math.drop(columns=["prog", "work"])
X_just_math.head()
```

Out[10]:

	const	math
45	1.000	41
108	1.000	41
15	1.000	44
67	1.000	42
153	1.000	40

In [11]:

```python
glm_poiss_math = sm.GLM(y, X_just_math, family=sm.families.Poisson()).fit()
print(glm_poiss_math.summary())
print("AIC score for Poisson GLM model: ", glm_poiss_math.aic)
print(compareGLM(null_model, glm_poiss_math))
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  200
Model:                            GLM   Df Residuals:                      198
Model Family:                 Poisson   Df Model:                            1
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -190.04
Date:                Mon, 10 Nov 2025   Deviance:                       204.02
Time:                        13:06:26   Pearson chi2:                     229.
No. Iterations:                     5   Pseudo R-squ. (CS):             0.3418
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -5.3335      0.591     -9.021      0.000      -6.492      -4.175
math           0.0862      0.010      8.902      0.000       0.067       0.105
==============================================================================
AIC score for Poisson GLM model:  384.07618503564413
=== GLM Model Comparison (Likelihood Ratio Test) ===
Log-likelihood (simple):   -293.600
Log-likelihood (complex):  -190.038
LR statistic:              207.124
Degrees of freedom diff:   1
p-value:                   0.00000
AIC (simple):              589.200
AIC (complex):             384.076
None
```

In [12]:

```python
X_math_prog = X.copy()
X_math_prog = X_math_prog.drop(columns=["work"])
X_math_prog.head()
```

Out[12]:

	const	prog	math
45	1.000	3	41
108	1.000	1	41
15	1.000	3	44
67	1.000	3	42
153	1.000	3	40

In [13]:

```python
glm_poiss_math_prog = sm.GLM(y, X_math_prog, family=sm.families.Poisson()).fit()
print(glm_poiss_math_prog.summary())
print(compareGLM(glm_poiss_math, glm_poiss_math_prog))
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  200
Model:                            GLM   Df Residuals:                      197
Model Family:                 Poisson   Df Model:                            2
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -189.75
Date:                Mon, 10 Nov 2025   Deviance:                       203.45
Time:                        13:06:26   Pearson chi2:                     227.
No. Iterations:                     5   Pseudo R-squ. (CS):             0.3437
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -5.5781      0.677     -8.242      0.000      -6.905      -4.252
prog           0.1233      0.163      0.755      0.450      -0.197       0.443
math           0.0861      0.010      8.984      0.000       0.067       0.105
==============================================================================
=== GLM Model Comparison (Likelihood Ratio Test) ===
Log-likelihood (simple):   -190.038
Log-likelihood (complex):  -189.753
LR statistic:              0.570
Degrees of freedom diff:   1
p-value:                   0.45021
AIC (simple):              384.076
AIC (complex):             385.506
None
```

In [14]:

```python
X_math_work = X.copy()
X_math_work = X_math_work.drop(columns=["prog"])
X_math_work.head()
```

Out[14]:

	const	math	work
45	1.000	41	19
108	1.000	41	31
15	1.000	44	3
67	1.000	42	34
153	1.000	40	0

In [15]:

```python
glm_poiss_math_work = sm.GLM(y, X_math_work, family=sm.families.Poisson()).fit()
print(glm_poiss_math_work.summary())
print(compareGLM(glm_poiss_math, glm_poiss_math_work))
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  200
Model:                            GLM   Df Residuals:                      197
Model Family:                 Poisson   Df Model:                            2
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -189.67
Date:                Mon, 10 Nov 2025   Deviance:                       203.29
Time:                        13:06:26   Pearson chi2:                     227.
No. Iterations:                     5   Pseudo R-squ. (CS):             0.3442
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -5.1722      0.620     -8.342      0.000      -6.387      -3.957
math           0.0851      0.010      8.712      0.000       0.066       0.104
work          -0.0073      0.009     -0.850      0.395      -0.024       0.010
==============================================================================
=== GLM Model Comparison (Likelihood Ratio Test) ===
Log-likelihood (simple):   -190.038
Log-likelihood (complex):  -189.675
LR statistic:              0.726
Degrees of freedom diff:   1
p-value:                   0.39402
AIC (simple):              384.076
AIC (complex):             385.350
None
```

In [16]:

```python
coef = glm_poiss_math.params['math']
irr = np.exp(coef)
pct = (irr - 1) * 100
print(f"math coef = {coef:.4f}, IRR = {irr:.3f}, => {pct:.2f}% change in expected awards per +1 math point")
```

```
math coef = 0.0862, IRR = 1.090, => 9.00% change in expected awards per +1 math point
```

Add your verbal summary and comments here (1p):

Selecting best Poisson GLM model

From the previous analysis, only one predictor was statistically significant: math. Starting with the math-only Poisson model and comparing it to the null model, the math-only model significantly improves the fit (higher log-likelihood, lower AIC). Although individual tests suggested that the other two predictors (prog and work) are not significant, we nevertheless tested adding them. Both the likelihood ratio test (p-value > 0.05) and AIC showed no improvement, so the simpler model with only math is preferred.

Interpretation of the math effect

The estimated coefficient for math is 0.0862. In a Poisson GLM with log link the effect is multiplicative: the incidence rate ratio (IRR) is exp(0.0862)=1.09. That means, an increase of one point in the math score corresponds to an expected increase of about 9% in the number of awards

Comparing Poisson GLM over OLS

The dependent variable (number of awards) represents counts, which violates the normality assumption required for OLS. The Poisson GLM is theoretically more appropriate because it models counts directly and ensures non-negative fitted values via the log link. Technically, the Poisson model (math-only) also performs better on the available fit metrics: AIC decreased from 536.79 (OLS/Gaussian) to 384.07 (Poisson), and the log-likelihood increased from −264.40 (OLS) to −190.04 (Poisson). These indicate a better fit for the Poisson GLM (math-only).

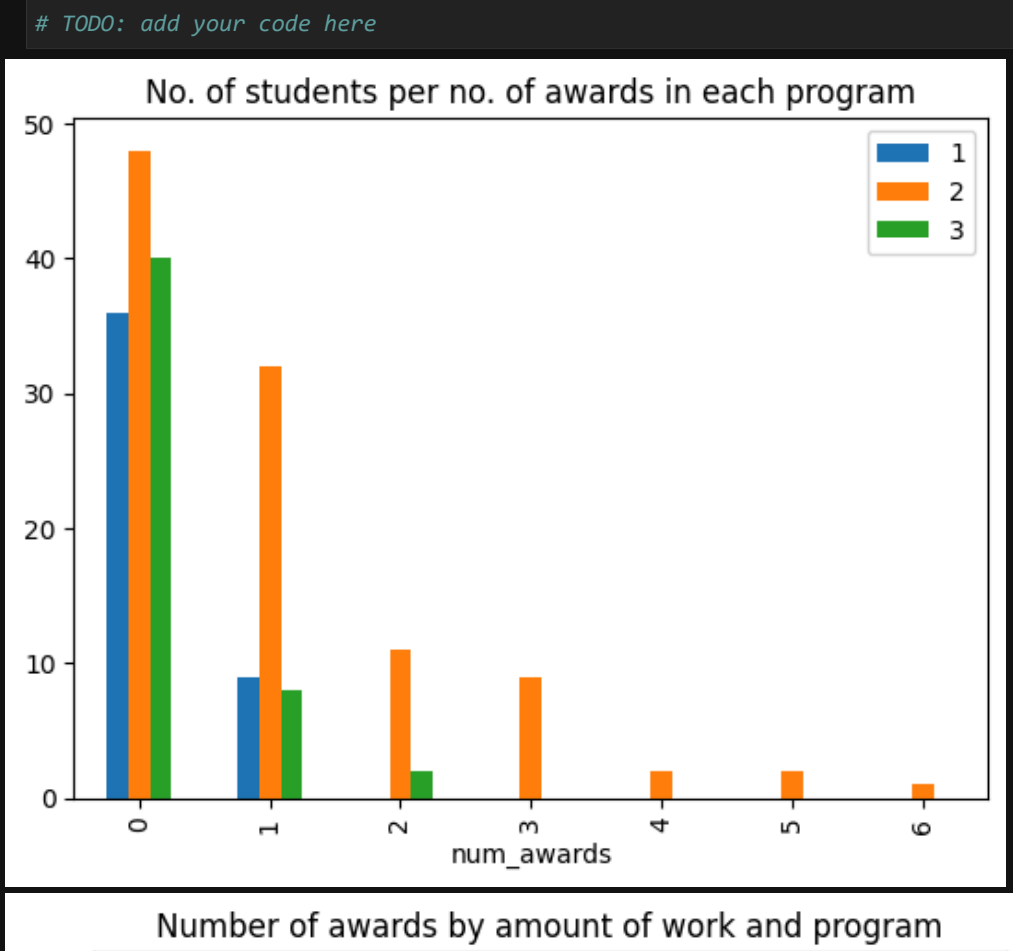
Feature transformations and final model

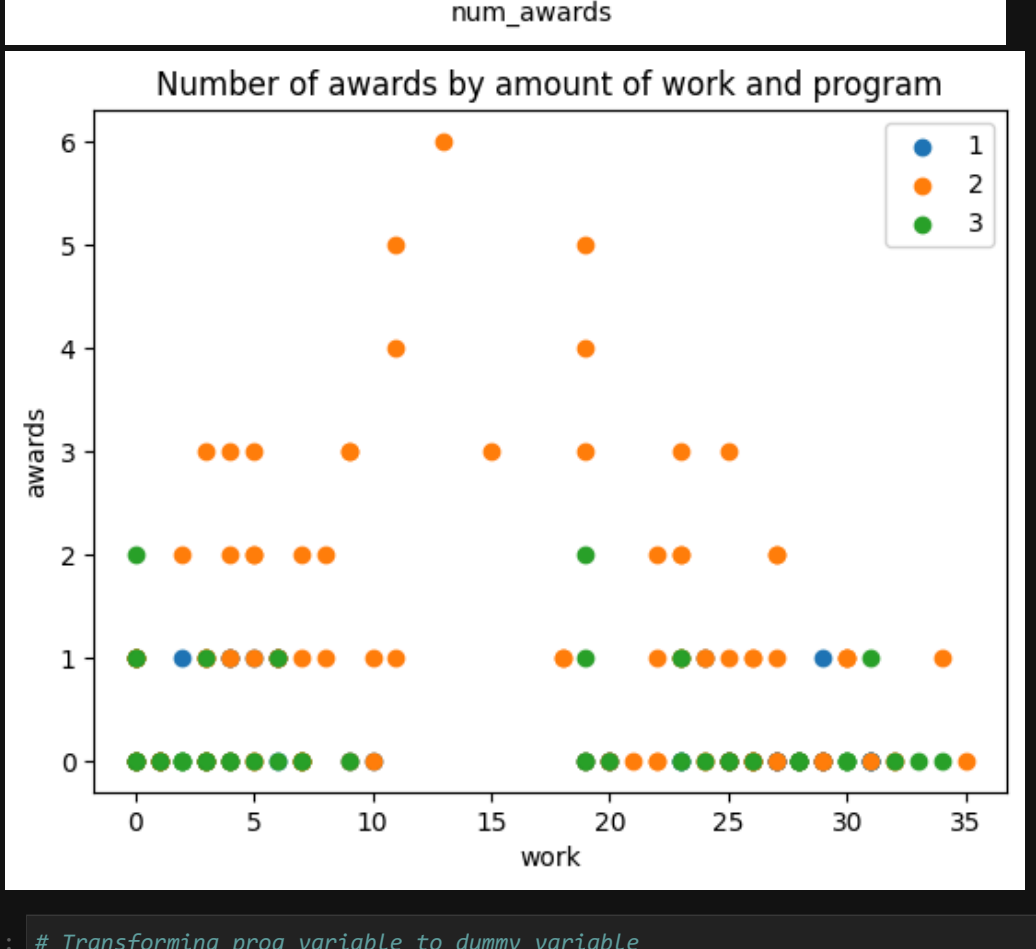
prog and *work* did not prove to be effective predictors so far. Visualize these predictors as well as their relationship with the outcome variable. Based on the observations, propose suitable transformations for them (or, justify that they are truly uninformative for prediction of *num_awards*) and implement them into the best model found by now. Use the *compareGLM()* function to validate that your new GLMs indeed improved over the simple one.

To achieve the same results as with R formulas (e.g. polynomial regression like *num_awards ~ poly(...)*) in Python you need to transform your features explicitly before passing them to your model. You do this by modifying the exog data in X. For tools to do polynomial or spline transformations refer for example to sklearn: SplineTransformer PolynomialFeatures

In [17]:

```python
cols = ["num_awards", "prog"]
df_transformed = df[cols].groupby(cols).size().reset_index(level=-1)
programs = pd.unique(df_transformed.prog).tolist()
pd.concat([
    df_transformed.loc[df_transformed.prog == p, 0]
    for p in programs
], axis=1).set_axis(programs, axis="columns").plot.bar()
plt.title("No. of students per no. of awards in each program")
plt.show()

for p in programs:
    plt.scatter(df.work.loc[df.prog == p], df.num_awards.loc[df.prog == p], label=p)
plt.title("Number of awards by amount of work and program")
plt.xlabel("work")
plt.ylabel("awards")
plt.legend()
plt.show()

# TODO: add your code here
```





In [18]:

```python
# Transforming prog variable to dummy variable
X_transformed_prog = X.copy()
X_transformed_prog = pd.get_dummies(X_transformed_prog, columns=["prog"], drop_first=True)
# Tranform dummy values to numeric
X_transformed_prog = X_transformed_prog.astype(float)
X_transformed_prog.head()
```

Out[18]:

	const	math	work	prog_2	prog_3
45	1.000	41.000	19.000	0.000	1.000
108	1.000	41.000	31.000	0.000	0.000
15	1.000	44.000	3.000	0.000	1.000
67	1.000	42.000	34.000	0.000	1.000
153	1.000	40.000	0.000	0.000	1.000

In [19]:

```python
X_transformed_no_work = X_transformed_prog.copy()
X_transformed_no_work = X_transformed_no_work.drop(columns=["work"])
X_transformed_no_work.head()
```

Out[19]:

	const	math	prog_2	prog_3
45	1.000	41.000	0.000	1.000
108	1.000	41.000	0.000	0.000
15	1.000	44.000	0.000	1.000
67	1.000	42.000	0.000	1.000
153	1.000	40.000	0.000	1.000

In [20]:

```python
glm_poiss_no_work = sm.GLM(y, X_transformed_no_work, family=sm.families.Poisson()).fit()
print(glm_poiss_no_work.summary())
print(compareGLM(glm_poiss_math_work, glm_poiss_no_work))
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  200
Model:                            GLM   Df Residuals:                      196
Model Family:                 Poisson   Df Model:                            3
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -182.75
Date:                Mon, 10 Nov 2025   Deviance:                       189.45
Time:                        13:06:27   Pearson chi2:                     212.
No. Iterations:                     5   Pseudo R-squ. (CS):             0.3881
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -5.2471      0.658     -7.969      0.000      -6.538      -3.957
math           0.0702      0.011      6.619      0.000       0.049       0.091
prog_2         1.0839      0.358      3.025      0.002       0.382       1.786
prog_3         0.3698      0.441      0.838      0.402      -0.495       1.234
==============================================================================
=== GLM Model Comparison (Likelihood Ratio Test) ===
Log-likelihood (simple):   -189.675
Log-likelihood (complex):  -182.752
LR statistic:              13.845
Degrees of freedom diff:   1
p-value:                   0.00020
AIC (simple):              385.350
AIC (complex):             373.505
None
```

In [21]:

```python
from sklearn.preprocessing import PolynomialFeatures

# Transform work variable as polynomial
X_transformed_full = X_transformed_prog.copy()
poly = PolynomialFeatures(degree=2, include_bias=False)
work_poly = poly.fit_transform(df[["work"]])
X_transformed_full["work"] = work_poly[:, 0]
X_transformed_full["work2"] = work_poly[:, 1]
X_transformed_full.head()
```

Out[21]:

	const	math	work	prog_2	prog_3	work2
45	1.000	41.000	19.000	0.000	1.000	361.000
108	1.000	41.000	31.000	0.000	0.000	961.000
15	1.000	44.000	3.000	0.000	1.000	9.000
67	1.000	42.000	34.000	0.000	1.000	1156.000
153	1.000	40.000	0.000	0.000	1.000	0.000

In [22]:

```python
glm_poiss_full_trans = sm.GLM(y, X_transformed_full, family=sm.families.Poisson()).fit()
print(glm_poiss_full_trans.summary())
print(compareGLM(glm_poiss_no_work, glm_poiss_full_trans))
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:                      y   No. Observations:                  200
Model:                            GLM   Df Residuals:                      194
Model Family:                 Poisson   Df Model:                            5
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -175.17
Date:                Mon, 10 Nov 2025   Deviance:                       174.29
Time:                        13:06:27   Pearson chi2:                     189.
No. Iterations:                     5   Pseudo R-squ. (CS):             0.4327
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -4.8927      0.681     -7.181      0.000      -6.228      -3.557
math           0.0575      0.011      5.257      0.000       0.036       0.079
work           0.1428      0.042      3.426      0.001       0.061       0.224
prog_2         0.9915      0.360      2.755      0.006       0.286       1.697
prog_3         0.3136      0.442      0.710      0.478      -0.552       1.179
work2         -0.0051      0.001     -3.631      0.000      -0.008      -0.002
==============================================================================
=== GLM Model Comparison (Likelihood Ratio Test) ===
Log-likelihood (simple):   -182.752
Log-likelihood (complex):  -175.170
LR statistic:              15.164
Degrees of freedom diff:   2
p-value:                   0.00051
AIC (simple):              373.505
AIC (complex):             362.340
None
```

Add your verbal summary and comments here (2p) :

From the first graph (No. of students per no. of awards in each program), it is clear that program affects the number of awards - students in program 2 tend to have higher award counts. Therefore, we included prog in the model and treated it as a categorical (dummy) variable. Comparing this model with the previous best model - math-only model, we observed an improvement - lower AIC, higher log-likelihood, and p-value < 0.05 in the likelihood ratio test.

From the second graph (number of awards by amount of work and program), it appears that work has a nonlinear relationship with awards. We therefore added a polynomial transformation for the work variable. After comparing this model (math + prog + polynomial work) with the previous one, the fit again improved - AIC decreased, log-likelihood increased, and p-value < 0.05, confirming that the transformation helps capture the relationship better.

Ablation study through cross-validation

Recap all the models considered previously and evaluate them through cross-validation. You can start with the most simple null model and gradually add the previously discussed improvements. See their role in terms of MAE, RMSE and other commonly used criteria. The procedure outlined below proposes to work with the preprepared *train_with_cv* function, you can only add more models to evaluate and compare.

In [37]:

```python
def test_glm_with_cv(X, y, model_type, model_name, model_family=sm.families.Gaussian(), n_folds=10):
    """
    You can use this function to test your GLM models in CV settings. If you want, you can edit
    this function, e.g. to have more comparison metrics, but justify it well in your commentary.

    :param X: your independent variables / features
    :param y: your response variable
    :param model_name: just for convenience you can name your models
    :param model_family: family of your GLM model (from statsmodels.families),
                         the default corresponds to an OLS model
    :param n_folds: number of folds to perform in your CV
    :return: returns a DataFrame with all the metrics for a particular model
    """
    assert model_type in ["lm", "glm"]
    data_metrics = {
        "MSE": lambda fit, x, y: mean_squared_error(y, x),
        "MAE": lambda fit, x, y: mean_absolute_error(y, x),
        "R2": lambda fit, x, y: r2_score(y, x),
        "LogLikelihood": lambda fit, x, y: fit.family.loglike_obs(endog=y, mu=x).sum(),
    }
    model_metrics = {
        "AIC": lambda fit: fit.aic,
        "BIC": lambda fit: fit.bic_llf,
    }

    skf = KFold(n_splits=n_folds, shuffle=True, random_state=0)
    metrics_results = defaultdict(list)
    for train_idxs, test_idxs in skf.split(X, y):
        model = sm.GLM(endog=y[train_idxs], exog=X[train_idxs], family=model_family)
        fit = model.fit()

        cmp = fit, y[test_idxs], fit.predict(X[test_idxs])
        for name, m in data_metrics.items():
            metrics_results[name].append(m(*cmp))

        for name, m in model_metrics.items():
            metrics_results[name].append(m(fit))

    res = pd.DataFrame({
        "Mean": [pd.Series(m).mean() for _, m in metrics_results.items()],
        "Sd": [pd.Series(m).std() for _, m in metrics_results.items()],
    }, index=[n for n, _ in metrics_results.items()])

    return res
```

In [38]:

```python
# Here you have an example of the null lm model trained and tested with CV

# the sm.add_constant prepared a column "const" of ones for the intercept estimation
print("Mean Model Resuts")
print(test_glm_with_cv(
    X = X.const.to_numpy(),
    y = df.num_awards.to_numpy(),
    model_type = "lm",
    model_name="mean_model"
))

# TODO: add your code here
print("")
print("--------------------------------------------------------------------------------------")
print("Ordinary linear regression model resuts")
print(test_glm_with_cv(
    X = X.to_numpy(),
    y = df.num_awards.to_numpy(),
    model_type = "lm",
    model_name="ols_model"
))

print("")
print("--------------------------------------------------------------------------------------")
print("Generalized linear model with math feature and without transformations resuts")
print(test_glm_with_cv(
    X = X_just_math.to_numpy(),
    y = df.num_awards.to_numpy(),
    model_type = "glm",
    model_name="math_poiss_model",
    model_family=sm.families.Poisson()
))

print("")
print("--------------------------------------------------------------------------------------")
print("Generalized linear model with math and prog as dummy variable and without work resuts")
print(test_glm_with_cv(
    X = X_transformed_no_work.to_numpy(),
    y = df.num_awards.to_numpy(),
    model_type = "glm",
    model_name="poiss_model_2",
    model_family=sm.families.Poisson()
))

print("")
print("--------------------------------------------------------------------------------------")
print("Generalized linear model with math, prog as dummy variable and wih work as polynomial resuts")
print(test_glm_with_cv(
    X = X_transformed_full.to_numpy(),
    y = df.num_awards.to_numpy(),
    model_type = "glm",
    model_name="poiss_model_3",
    model_family=sm.families.Poisson()
))
```

```
Mean Model Resuts
                 Mean      Sd
MSE             1.115   0.639
MAE             0.785   0.142
R2            >1.e+05 >1.e+05
LogLikelihood -29.531   6.386
AIC           530.034  11.757
BIC           533.227  11.757

--------------------------------------------------------------------------------------
Ordinary linear regression model resuts
                 Mean     Sd
MSE             0.866  0.485
MAE             0.639  0.175
R2             -2.630  2.549
LogLikelihood -27.044  4.849
AIC           483.083 11.720
BIC           495.855 11.720

--------------------------------------------------------------------------------------
Generalized linear model with math feature and without transformations resuts
                 Mean    Sd
MSE             0.849 0.469
MAE             0.628 0.137
R2             -2.621 2.456
LogLikelihood >1.e+05   NaN
AIC           345.747 9.062
BIC           352.133 9.062

--------------------------------------------------------------------------------------
Generalized linear model with math and prog as dummy variable and without work resuts
                 Mean    Sd
MSE             0.799 0.426
MAE             0.600 0.135
R2             -1.757 1.814
LogLikelihood >1.e+05   NaN
AIC           336.483 8.828
BIC           349.255 8.828

--------------------------------------------------------------------------------------
Generalized linear model with math, prog as dummy variable and wih work as polynomial resuts
                 Mean    Sd
MSE             0.717 0.380
MAE             0.588 0.139
R2             -0.765 0.968
LogLikelihood >1.e+05   NaN
AIC           326.648 7.705
BIC           345.806 7.705
```

```
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\pandas\core\nanops.py:1016: RuntimeWarning: invalid value encountered in subtract
  sqr = _ensure_numeric((avg - values) ** 2)
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\pandas\core\nanops.py:1016: RuntimeWarning: invalid value encountered in subtract
  sqr = _ensure_numeric((avg - values) ** 2)
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\statsmodels\genmod\families\family.py:477: RuntimeWarning: divide by zero encountered in log
  return var_weights / scale * (endog * np.log(mu) - mu -
c:\Users\gavul\Desktop\cvut_programming\CTU_SAN\venv\Lib\site-packages\pandas\core\nanops.py:1016: RuntimeWarning: invalid value encountered in subtract
  sqr = _ensure_numeric((avg - values) ** 2)
```

Add your verbal summary and comments here (1p):

- Cross-validation confirms the same trend as we observed before: model performance consistently improves as we add meaningful predictors, transformations and try Poisson GLM - null model < OLS < Poisson (math only) < Poisson (math + prog dummy) < Poisson (math + prog dummy + polynomial work).
- Metrics MAE, RMSE and both AIC/BIC decrease with each step, which indicates a better fit.
- This shows that each improvement contributes to a more accurate and appropriate model for count data.

Feedback

Was some part of the notebook unclear, would any topic need more attention during the tutorials? If you want to leave us feedback on the assignment, we would be happy to hear it. Here is your space: